

40.016 The Analytics Edge

Classification And Regression Tree (Cart) and Random Forest (RF)

Yihan Du

Engineering Systems and Design (ESD)
Singapore University of Technology and Design (SUTD)

Summer 2026

Contact

Yihan Du

Email: yihan_du@sutd.edu.sg

Office: 1.702.12

For consultation, just email me and set up a time

Teaching Assistants:

- Fuyang Wei: fuyang_wei@mymail.sutd.edu.sg
- Gee Nee Liaw: genee_liaw@sutd.edu.sg
- Adharsha Sam Edwin Sam Devahi: adharsha_devahi@sutd.edu.sg

Important Information

Public Holiday Makeup Lessons for CS02:

- **5 Aug 2026 (Wed), 11:30am - 1:30pm, TT2/TT3**
(30min later than the original schedule of 11am – 1pm)
- **7 Aug 2026 (Fri), 6pm - 8pm, CC10**
(replaces the original schedule of 10 Aug 2026, Mon, 1pm – 3pm)

One Homework:

- Tentative schedule: released after teaching CART and Random Forests (Week 9)
- Duration: one week

Important Information

Project (tentative schedule):

- Competition on Kaggle: the end of Week 10 - the end of Week 11
- Report and code: the end of Week 11 - the start of Week 13

Final Exam: **19 Aug 2026 (Wed) at 9am**

- Duration: 1 hour 20 minutes
- Written exam. All questions are multiple-choice questions (choose one correct answer from four options)
- Can bring a one-page, double-sided A4 reference sheet (handwritten or typed)
- Focus on the understanding of the concepts and models taught in class, instead of the R programming language

Overview

Topics:

- Decision Trees and Random Forests
- Clustering and Recommendation Systems
- Matrix Factorization
- Censored Data Analysis
- Optimization

Forecasting Supreme Court Decisions

What is the Supreme Court? See: [Youtube](#)

Forecasting Supreme Court Decisions

What is the Supreme Court? See: [Youtube](#)

Main Takeaway:

- The highest court in the United States
- 9 justices appointed by the US President
- Lifetime tenure
- The court handles ~ 80 cases per year
- A decision happens when the majority agrees on an outcome

Forecasting Supreme Court Decisions

What is the Supreme Court? See: [Youtube](#)

Categories for case selection:

- Cases of national importance
- Lower court claimed that a federal law breaks the Constitution
- Resolve split decision
- Landmark US Supreme Court cases. See: [American Bar Association \(ABA\) website](#)

Forecasting Supreme Court Decisions

This is a study published by Martin et al. (2004), who:

- Used data spanning 1994-2001 (longest period with the same justices)
- Compared predictions (for the year 2002) made by legal experts and statistical models
- Found very interesting results:
 - Accuracy on the entire court decision: models, 75%; experts, 59.1%
 - Accuracy at the individual justice level: models, 66.7%; experts, 67.9%

Forecasting Supreme Court Decisions

Data:

- 623 observations, 20 variables
- Response variable (predictand): decision result
- Input variables (predictors):
 - petit: type of petitioner (e.g., US government, employee, injured person)
(The petitioner is the party that is unhappy with the lower court's decision and asks the Supreme Court to review.)
 - respon: type of respondent (e.g., state government, employer)
(The respondent is the party that wins the original case and wants to defend the lower court's decision.)
 - circuit: the circuit court where the case originated (e.g., 2nd Circuit, 9th Circuit)

Forecasting Supreme Court Decisions

Data:

- Input variables (predictors):
 - unconst: whether the petitioner claimed that the practice breaks the Constitution
 - lctdir: ideological direction of the lower court (liberal or conservative)
(liberal: tend to support individual rights; conservative: tend to support traditional values.)
 - issue: issue area of the case (e.g., criminal law, economic regulation)

Agenda

- Classification and Regression Trees
 - Regression Tree
 - Classification Tree
 - Pruning A Tree
- Bagging and Random Forests
 - Bootstrapping
 - Bagging
 - Random Forests

Classification and Regression Trees

Decision Tree

Goal: predict a response Y from predictors $X_1, \dots, X_p$

Main Idea: **partition** the space of predictor variables into a number of simple regions, and then use **simple rules** in each partitioned space to make predictions.

Simple rules:

- Regression tree: take the mean of the predicted values among the observations in this region
- Classification tree: take the mode (majority vote) of the predicted class among the observations in this region

Example – Regression Tree

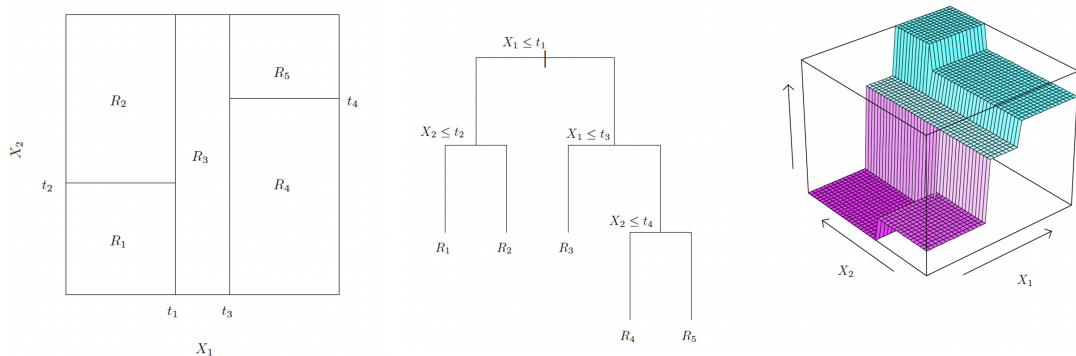


Figure: Regression tree example (Source: James et al., 2014)

Example – Regression Tree

How to read the model in the previous example?

Check if $X_1 \leq t_1$:

- If yes, check if $X_2 \leq t_2$;
 - If yes, we are in region R_1 .
 - If no, we are in region R_2 .
- If no, check if $X_1 \leq t_3$:
 - If yes, we are in region R_3 .
 - If no, check if $X_2 \leq t_4$:
 - If yes, we are in region R_4 ;
 - If no, we are in region R_5 .

Example – Classification Tree

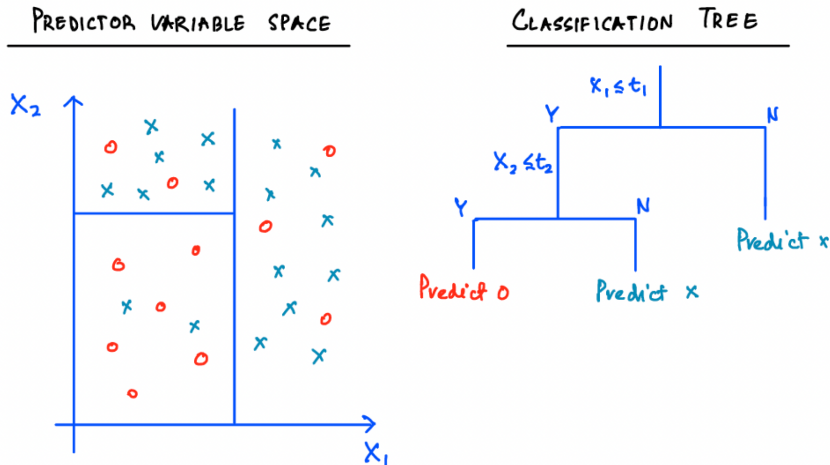


Figure: Classification tree example –predict “x” or “o” (Source: Teaching notes)

Finding The Tree (Regression)

Two main steps:

- (1) Partition the predictor space into J distinct and non-overlapping regions $(R_1, R_2, \dots, R_J)$
- (2) For every observation that falls into the j -th region R_j , we make the same prediction $\bar{y}_{R_j}$. Specifically, we take the mean of the response values for the training observations in R_j ¹:

$$\bar{y}_{R_j} = \text{ave}(y_i \mid x_i \in R_j).$$

¹The choice of using the mean of y_i in region R_j is based on the criterion minimization of the sum of squared errors.

Finding The Tree (Regression)

The problem of partitioning the predictor space into J regions (i.e., Step (1) in the previous slide) can be formulated as follows:

$$\min_{R_1, \dots, R_J \in \mathcal{R}} \sum_{j=1}^J \sum_{i \in R_j} (y_{ij} - \bar{y}_{R_j})^2,$$

- $\mathcal{R}$: the set of all possible partitions such that $R_i \cap R_j = \emptyset, \forall i \neq j$ and $R_1 \cup R_2 \cup \dots \cup R_J =$ the full predictor space
- y_{ij} : response of the i -th observation in R_j
- $\bar{y}_{R_j}$: mean of all observations in R_j

Finding The Tree (Regression)

The problem of partitioning the predictor space into J regions (i.e., Step (1) in the previous slide) can be formulated as follows:

$$\min_{R_1, \dots, R_J \in \mathcal{R}} \sum_{j=1}^J \sum_{i \in R_j} (y_{ij} - \bar{y}_{R_j})^2,$$

- $\mathcal{R}$: the set of all possible partitions such that $R_i \cap R_j = \emptyset, \forall i \neq j$ and $R_1 \cup R_2 \cup \dots \cup R_J = \text{the full predictor space}$
- y_{ij} : response of the i -th observation in R_j
- $\bar{y}_{R_j}$: mean of all observations in R_j

The problem is **Computationally Infeasible**.

Finding The Tree (Regression)

Recursive Binary Splitting (by Breiman [1984])

- Top-down: Begin at the top of the tree, and then successively split the predictor space into two new branches.
- Greedy: Don't look ahead or pick a split that will lead to a better tree in some future step.

Finding The Tree (Regression)

Recursive Binary Splitting

- Start with all observations in a single region.
- Consider all predictors $X_1, \dots, X_p$ and all split points s , and choose the predictor and split point s.t. the resulting partition has the lowest sum of squared errors (SSE).
More precisely, we define the following half-planes:

$$R_1(j, s) = \{X \mid X_j < s\} \text{ and } R_2(j, s) = \{X \mid X_j \geq s\}, \text{ where } j \in \{1, \dots, p\}$$

We seek the value of j and s that minimizes the SSE:

$$\min_{(j,s)} \sum_{i: x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2.$$

Finding The Tree (Regression)

Recursive Binary Splitting

- Start with all observations in a single region.
- Consider all predictors $X_1, \dots, X_p$ and all split points s , and choose the predictor X_j and split point s s.t. the resulting partition has the lowest SSE.
- Repeat the process for each split region. The process continues until an exit condition is met (e.g., reach the minimum number of points in a region, no meaningful improvement).

Finding The Tree (Classification)

A classification tree is built in almost the same manner as a regression tree, except:

- For each region, the prediction is the *majority* class among the training observations in this region (use the majority vote instead of the average value).
- When learning a tree, we use a measure of *impurity*, instead of SSE. A split is considered pure if every branch contains observations belonging to only one class.

Finding The Tree (Classification)

Measures of impurity

(1) Classification error rate:

$$E = 1 - \max_{k \in \{1, \dots, K\}} (p_{mk})$$

where p_{mk} is the proportion of observations in the m -th region from the k -th class and K is the total number of classes.

Finding The Tree (Classification)

(2) Gini index:

$$G = \sum_{k=1}^K p_{mk} (1 - p_{mk}) = 1 - \sum_{k=1}^K p_{mk}^2.$$

When $K = 2$, G varies between 0 and 0.5. Specifically, when $p_{m1} = p_{m2} = 0.5$, G achieves the maximum value $1 - (0.5^2 + 0.5^2) = 0.5$.

Finding The Tree (Classification)

(3) Entropy (or Information Gain) given by

$$D = - \sum_{k=1}^K p_{mk} \cdot \log_2(p_{mk}),$$

When $K = 2$, D varies between 0 and 1. Specifically, when $p_{m1} = p_{m2} = 0.5$, D achieves the maximum value $-2 * 0.5 * \log_2(0.5) = 1$.

Finding The Tree (Classification)

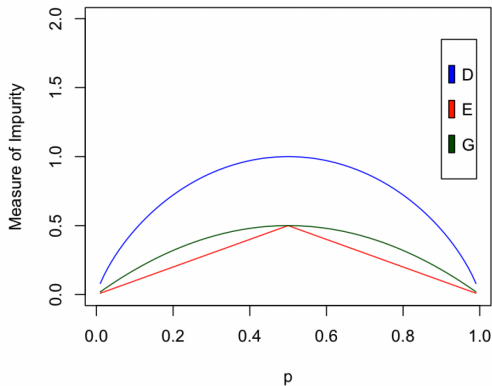


Figure: Measure of impurity (Source: Teaching Notes)

Here the figure plots the case $K = 2$, and the X-axis represents $p = p_{m1}$ ($p_{m2} = 1 - p$).

Differences Between Regression Tree and Classification Tree

	Regression Tree	Classification Tree
Outcome	a value	a class
Splitting Criteria	sum of squared errors	Gini impurity or entropy
Goal	predicted value close to real value	achieve high class purity
Example	predict price of a new product	predict if an email is spam

Table: Regression Tree Versus Classification Tree

Recap: Confusion Matrix, ROC Curve and AUC

A table that summarizes the performance of a classification model.

	Predicted No	Predicted Yes
Actual No	TN	FP
Actual Yes	FN	TP

False positive rate: $FPR = \frac{FP}{FP+TN}$

True positive rate: $TPR = \frac{TP}{TP+FN}$

Accuracy: $acc = \frac{TP+TN}{Total}$

ROC curve: (FPR, TPR), classification threshold varies from 1 to 0

AUC: the area under the ROC curve

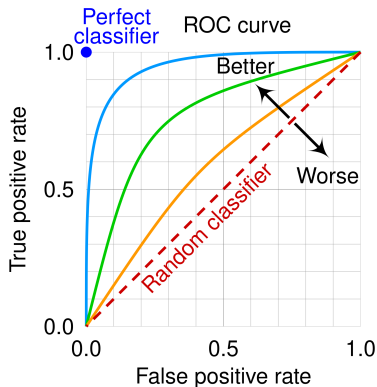


Figure: ROC Curve (Source: Wikipedia)

R Implementation

How can we implement CARTs in R?

- use the function `rpart`, implemented in the package `. . . rpart`
- `rpart(formula, data, method, control, ...)`

Pruning a Tree

- CART is likely to build complex trees that overfit the training data.
- A smaller tree (with fewer region $R_1, R_2, \dots, R_J$) may be more robust but biased.

Pruning a Tree

- **Bias:** the error introduced by approximating a function f with a simpler function $\hat{f}$
- **Variance:** measures how the estimation would change if we used a different training dataset

Bias-variance trade-off

- More complex model $\hat{f} \Rightarrow$ Low bias and high variance, since the model captures many features of the data;
- Simpler model $\hat{f} \Rightarrow$ High bias and low variance.

Pruning a Tree

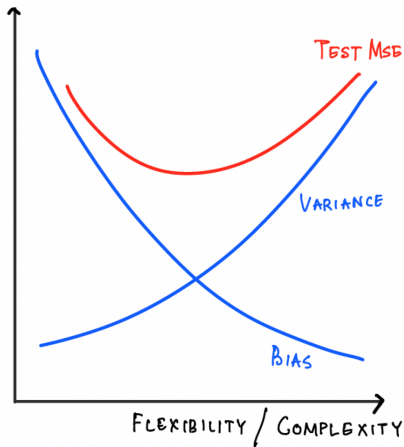


Figure: Bias-variance trade-off (Source: Teaching notes)

Pruning a Tree

Idea

- (1) Grow a large tree T_0
- (2) Prune it to obtain a subtree

Cross-Validation Error

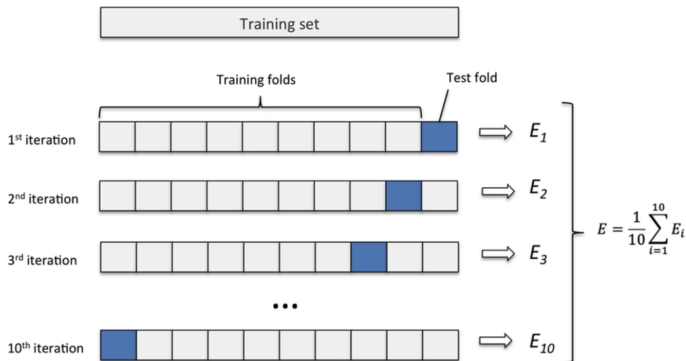


Figure: 10-fold cross-validation (Source: Research gate)

Pruning a Tree

It might not be feasible to calculate the cross-validation error for all possible pruned trees.

Cost complexity pruning (or weakest link pruning)

For a regression tree, given a tuning parameter $\alpha \geq 0$, we find the sub-tree $T \subseteq T_0$ of the original tree (T_0) such that:

$$\min_{T \subseteq T_0} \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|,$$

where $|T|$ is the number of leaves (terminal nodes) in the tree.

- As α increases, we pay a price for building a tree with many leaves.
- The above expression is reminiscent of the LASSO, i.e., linear regression with L1 regularization penalty (the first part of this course).

Pruning a Tree

(Full) algorithm for building a Regression Tree:

- (1) Use recursive binary splitting to grow a large tree on the training data.
- (2) Use cost complexity pruning to obtain different sub-trees for different values of α .
- (3) Use k -fold cross-validation to choose the best value of α .
- (4) Return the sub-tree from Step 2 that corresponds to the chosen value of α .

Pruning a Tree

For a classification tree, we follow the same procedure (with the model error calculated with a measure of impurity). We find the sub-tree $T \subseteq T_0$ of the original tree (T_0) such that

$$\min_{T \subseteq T_0} \sum_{m=1}^{|T|} E_m + \alpha |T|,$$

where E_m denotes the impurity measure of the m -th terminal node, quantifying its classification error.

R Implementation

The function `rpart` implemented in the package `rpart` already conducts pruning with a 10-fold cross-validation by default.

`cp` denotes the cost-complexity parameter α . Check using: `printcp(model)`

Then you may use:

```
prune(model,opt-cp)
```

Advantages and Disadvantages of CARTs

Pros:

- Interpretable
- Can be displayed graphically
- Might more closely mirror human decision-making
- Can handle qualitative predictors (that take no continuous values)
- No assumptions on the relationship between input and output (response) variables

Cons:

- They are not very accurate
- Not robust (high variance: small changes in training data lead to large changes in the tree structure)

Linear Model (Logistic Regression) Versus CART

Linear models are better for data with simple, linear boundaries.

CART is better for data with complex or non-linear decision regions.

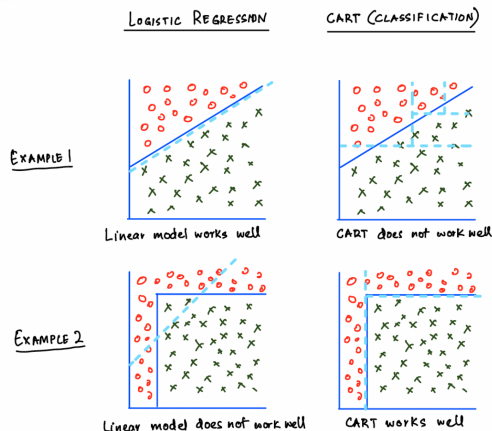


Figure: Trees versus linear models (Source: Teaching notes)

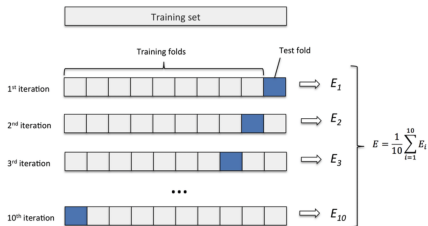
Bagging and Random Forests

Bootstrapping

- The Bootstrap is a resampling method used to quantify the uncertainty associated with a statistical learning method (or a given estimator).
- The term originates from the expression “to pull oneself up by one’s bootstraps”.
- Other resampling method introduced in this course?

Bootstrapping

- The Bootstrap is a resampling method used to quantify the uncertainty associated with a statistical learning method (or a given estimator).
- The term originates from the expression “to pull oneself up by one’s bootstraps”.
- Other resampling method introduced in this course?
 - Cross-validation error



Bootstrapping

Example

- We want to invest an amount of money in two financial assets, each yielding a random return per unit investment X and Y respectively.
- We will invest a fraction α of our money in X , and the remaining $1 - \alpha$ in Y .
- We want to choose the value of α that minimizes the total risk, measured by the variance per unit investment, $\text{Var}(\alpha X + (1 - \alpha)Y)$.

Bootstrapping

Example

- We want to invest an amount of money in two financial assets, each yielding a random return per unit investment X and Y respectively.
- We will invest a fraction α of our money in X , and the remaining $1 - \alpha$ in Y .
- We want to choose the value of α that minimizes the total risk, measured by the variance per unit investment, $\text{Var}(\alpha X + (1 - \alpha)Y)$.
- The value of α that minimizes the risk is

$$\alpha^* = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}},$$

where $\sigma_X^2 = \text{Var}(X)$, $\sigma_Y^2 = \text{Var}(Y)$ and $\sigma_{XY} = \text{Cov}(X, Y)$.

Bootstrapping

Example

- The values of σ_X^2 , σ_Y^2 , and σ_{XY} are unknown.
- Calculate estimates $\hat{\sigma}_X^2$, $\hat{\sigma}_Y^2$, and $\hat{\sigma}_{XY}$ using a dataset with observations of X and Y .
- Then, estimate α^* as

$$\hat{\alpha}^* = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}.$$

Bootstrapping

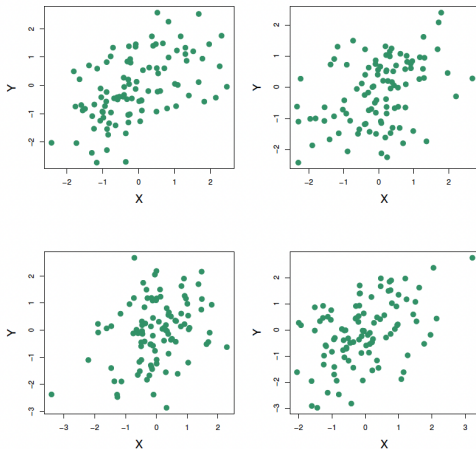


Figure: Several simulated returns for (X, Y) (Source: James et al., 2014). Generate a new dataset for each subfigure (simulation).

Bootstrapping

Example

Repeat this process 1,000 times, so as to get a better estimate of α :

- Suppose $\sigma_X^2 = 1$, $\sigma_Y^2 = 1.25$, and $\sigma_{XY} = 0.50$, so the true value of α^* is 0.60.
- The mean over all 1,000 estimates for $\hat{\alpha}$ is

$$\bar{\alpha} = \frac{1}{1000} \sum_{r=1}^{1000} \hat{\alpha}_r = 0.5996,$$

which is close to 0.60.

- The standard deviation of the 1,000 estimates for $\hat{\alpha}$ is

$$\sqrt{\frac{1}{1000 - 1} \sum_{r=1}^{1000} (\hat{\alpha}_r - \bar{\alpha})^2} = 0.083$$

Bootstrapping

In practice, can we implement this process?

- No, because we cannot keep generating new samples.
- Bootstrapping mimics this process: we obtain distinct datasets by *repeatedly sampling observations from a given data set with replacement*.
- Each of these bootstrap datasets is created by sampling with replacement, and has *the same size as our original dataset*.

Bootstrapping

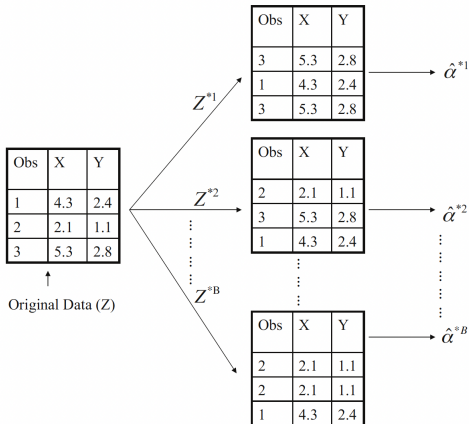


Figure: An illustration of the bootstrap approach (Source: James et al., 2014)

Bootstrapping

- Denoting the first bootstrap data set by Z^{*1} , we use Z^{*1} to produce a new bootstrap estimate for α which we call $\hat{\alpha}^{*1}$.
- The procedure is repeated B times (for example, 100 or 1,000), in order to produce B different bootstrap datasets ($Z^{*1}, Z^{*2}, \dots, Z^{*B}$), and B corresponding estimates $\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}$.
- Estimate α^* by

$$\hat{\alpha}^* = \frac{1}{B} \sum_{r=1}^B \hat{\alpha}^{*r}.$$

- Estimate the standard error by

$$SE_B(\hat{\alpha}^*) = \sqrt{\frac{1}{B-1} \sum_{r=1}^B (\hat{\alpha}^{*r} - \hat{\alpha}^*)^2}.$$

Bootstrapping

Difference between Cross-Validation and Bootstrapping

- Bootstrap resamples with replacement, while cross-validation resamples without replacement.
- The main goal of cross-validation is to measure, or generalize, the performance of a model, e.g., how good is my model likely to be on future data?
- Bootstrapping is used to estimate sampling distribution for a widespread range of statistics, e.g., how much uncertainty is there in this statistic or model estimate.

Bagging

Decision Trees suffer from high variance:

- Small changes in the training data can lead to a very different tree.

Bagging uses bootstrap resampling to reduce the variance of high-variance learners, such as Decision Trees, leading to more stable predictions and better predictive performance.

Bagging (or **Bootstrap aggregation**) combines predictions from many trees to reduce variance.

Bagging

How does it work?

- Given a set of n independent observations $Z_1, Z_2, \dots, Z_n$, each with variance σ^2 , the variance of the mean $\bar{Z}$ of observations is $\frac{\sigma^2}{n}$.
- Averaging a set of observations reduces variance.
- When learning a statistical model, we can therefore:
 - (1) Take many training sets from the population.
 - (2) Build a separate model using each training dataset.
 - (3) Average the resulting predictions.

Bagging

Formal steps for the algorithm:

- (1) Take repeated samples from the training dataset. We generate B different bootstrapped training datasets.
- (2) We train model $f^{*b}(x)$ on the b -th bootstrapped training dataset (repeat for all B datasets).
- (3) We average all the predictions as follows

$$f_{bag}(x) = \frac{1}{B} \sum_{b=1}^B f^{*b}(x).$$

This entire process is called **Bagging**.

Bagging

How can we estimate the prediction error of a bagged model?

- We could think about using each bootstrap dataset as our training sample, and the original sample as our validation sample.

Bagging

How can we estimate the prediction error of a bagged model?

- We could think about using each bootstrap dataset as our training sample, and the original sample as our validation sample.
- But each bootstrap sample has significant overlap with the original data. *Some samples appear in both training and validation datasets.* So this error estimation method will underestimate the true prediction error.
- We can fix the problem by using the out-of-bag (OOB) estimate.

Bagging

Out-of-Bag Error Estimation

- To estimate the test error of a bagged model, we can use the out-of-bag (OOB) observations:
 - To obtain the OOB prediction for the i -th observation: We *consider only the trees that do not include this observation in their bootstrap samples*, and average their predicted values (for regression) or take a majority vote (for classification).
 - Repeat the above step for all observations $i = 1, \dots, n$. Then, we can compute the overall OOB MSE or classification error.
- When B is sufficiently large, OOB error often provides a good alternative to cross-validation.

R Implementation

How can we implement Bagging in R?

- function bagging (package ipred)
- worked for Decision Trees (and some other algorithms, but not every algorithm)

Random Forests

Problems with using CART (vanilla)

- (1) CART suffers from high variance.
- (2) When building CART with Bagging, trees tend to be correlated.

Example:

- Suppose that the training dataset has a strong predictor and a few moderately strong predictors.
- Then, most of the bagged trees will use the strong predictor at the top split(s). As a result, trees will be correlated (similar), so we won't reduce variance much.

Random Forests

Idea

When building the trees, we consider a random sample of m ($m < p$) predictors from the full set of p predictors $X_1, \dots, X_p$ at each split:

- For each split, we consider only a subset of m predictors.
- On average, $p - m$ predictors are not considered, so other predictors will have a chance.
- This process decorrelates the trees.

Random Forests

Algorithm

- (1) Generate B bootstrapped training datasets.
- (2) For each bootstrapped dataset, train a Decision Tree.
 - At each split, use a subset of m predictors from the full set of p predictors.
- (3) Average the predicted values (for regression) or use the majority vote (for classification) from the B trees.

On the value of m

The fundamental difference between Bagging and Random Forests stands in the subset of predictors m :

- If $m = p$, then there is no difference between the two methods.
- Recommended values of m :
 - Regression: $m = p/3$
 - Classification: $m = \sqrt{p}$.
- Note that these values were found experimentally, so there is no theoretical guarantee that they will provide the best performance on all datasets.

Random Forests

Hyperparameter Tuning

It is common practice to explore the effect of the hyperparameters value on the performance of Random Forests.

- Number of trees, B .
- Number of predictors used at each split, $m \leq p$.

No optimization routines to find their values. Typically: grid search.

R Implementation

How can we implement Random Forests in R?

- function `randomForest`, implemented in the package `. . . randomForest`

References

- Martin et al. (2004) Competing approaches to predicting supreme court decision making. *Perspectives on Politics*, 2 (4), 761-767.
- James et al. (2014) *An Introduction to Statistical Learning with Applications in R*, Springer, 2014. Chapter 5.2, 8.1 and 8.2.